



Search

Write



★ Member-only story

YOLO — You only look once, real time object detection explained



Manish Chablani · Follow

Published in Towards Data Science · 8 min read · Aug 20, 2017



2.3K



8



Original paper (CVPR 2016. OpenCV People's Choice Award)

<https://arxiv.org/pdf/1506.02640v5.pdf>

YOLOv2: <https://arxiv.org/pdf/1612.08242v1.pdf>

Biggest advantages:

- Speed (45 frames per second — better than realtime)

- Network understands generalized object representation (This allowed them to train the network on real world images and predictions on artwork was still fairly accurate).
- faster version (with smaller architecture) — 155 frames per sec but is less accurate.
- open source: <https://pjreddie.com/darknet/yolo/>

High level idea:

Compared to other region proposal classification networks (fast RCNN) which perform detection on various region proposals and thus end up performing prediction multiple times for various regions in a image, Yolo architecture is more like FCNN (fully convolutional neural network) and passes the image (nxn) once through the FCNN and output is (mxm) prediction. This the architecture is splitting the input image in mxm grid and for each grid generation 2 bounding boxes and class probabilities for those bounding boxes. Note that bounding box is more likely to be larger than the grid itself. From paper:

We reframe object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities.

A single convolutional network simultaneously predicts multiple bounding boxes and class probabilities for those boxes. YOLO trains on full images and directly optimizes detection performance. This unified model has several benefits over traditional methods of object detection. First, YOLO is extremely fast. Since we frame detection as a regression problem we don't need a complex pipeline. We simply run our neural network on a new image at test time to predict detections. Our base network runs at 45 frames per second with no batch processing on a Titan X GPU and a fast version runs at more than 150 fps. This means we can process streaming video in real-time with less than 25 milliseconds of latency.

Second, YOLO reasons globally about the image when making predictions. Unlike sliding window and region proposal-based techniques, YOLO sees the entire image during training and test time so it implicitly encodes contextual information about classes as well as their appearance. Fast R-CNN, a top detection method, mistakes background patches in an image for objects because it can't see the larger context. YOLO makes less than half the number of background errors compared to Fast R-CNN.

Third, YOLO learns generalizable representations of objects. When trained on natural images and tested on artwork, YOLO outperforms top detection methods like DPM and R-CNN by a wide margin. Since YOLO is highly generalizable it is less likely to break down when applied to new domains or unexpected inputs.

Our network uses features from the entire image to predict each bounding box. It also predicts all bounding boxes across all classes for an image simultaneously. This means our network reasons globally about the full image and all the objects in the image. The YOLO design enables end-to-end training and realtime speeds while maintaining high average precision.

Our system divides the input image into an $S \times S$ grid. If the center of an object falls into a grid cell, that grid cell is responsible for detecting that object.

*Each grid cell predicts B bounding boxes and confidence scores for those boxes. These confidence scores reflect how confident the model is that the box contains an object and also how accurate it thinks the box is that it predicts. Formally we define confidence as $\text{Pr}(\text{Object}) * \text{IOU}$. If no object exists in that cell, the confidence scores should be zero. Otherwise we want the confidence score to equal the intersection over union (IOU) between the predicted box and the ground truth*

Each bounding box consists of 5 predictions: x , y , w , h , and confidence. The (x, y) coordinates represent the center of the box relative to the bounds of the grid cell. The width and height are predicted relative to the whole image. Finally the confidence prediction represents the IOU between the predicted box and any ground truth box. Each grid cell also predicts C conditional class probabilities, $\text{Pr}(\text{Class}_i | \text{Object})$. These probabilities are conditioned on the grid cell containing

an object. We only predict one set of class probabilities per grid cell, regardless of the number of boxes B .

At test time we multiply the conditional class probabilities and the individual box confidence predictions,

$$\text{Pr}(\text{Class}_i | \text{Object}) * \text{Pr}(\text{Object}) * \text{IOU} = \text{Pr}(\text{Class}_i) * \text{IOU}$$

, which gives us class-specific confidence scores for each box. These scores encode both the probability of that class appearing in the box and how well the predicted box fits the object

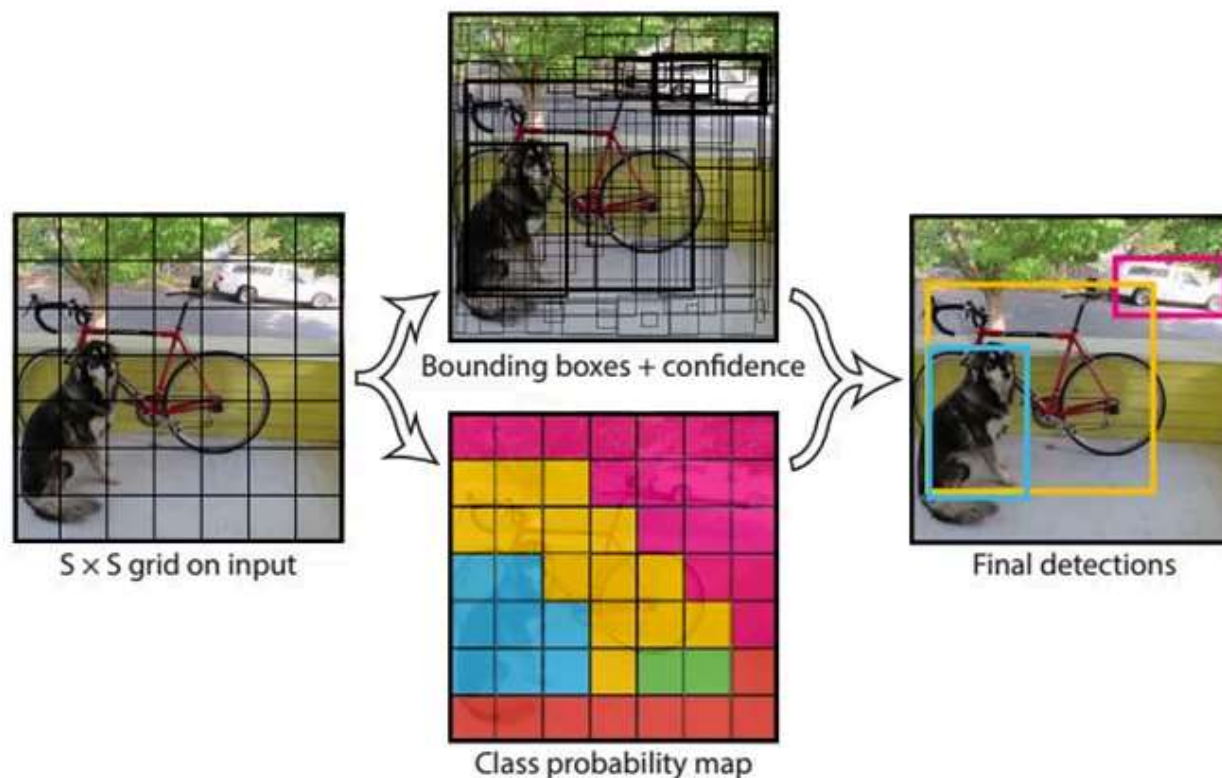


Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

Network Architecture and Training:

Changes to loss functions for better results is interesting. Two things stand out:

1. Differential weight for confidence predictions from boxes that contain object and boxes that don't contain object during training.
2. predict the square root of the bounding box width and height to penalize error in small object and large object differently.

Our network has 24 convolutional layers followed by 2 fully connected layers. Instead of the inception modules used by GoogLeNet, we simply use 1×1 reduction layers followed by 3×3 convolutional layers

Fast YOLO uses a neural network with fewer convolutional layers (9 instead of 24) and fewer filters in those layers. Other than the size of the network, all training and testing parameters are the same between YOLO and Fast YOLO.

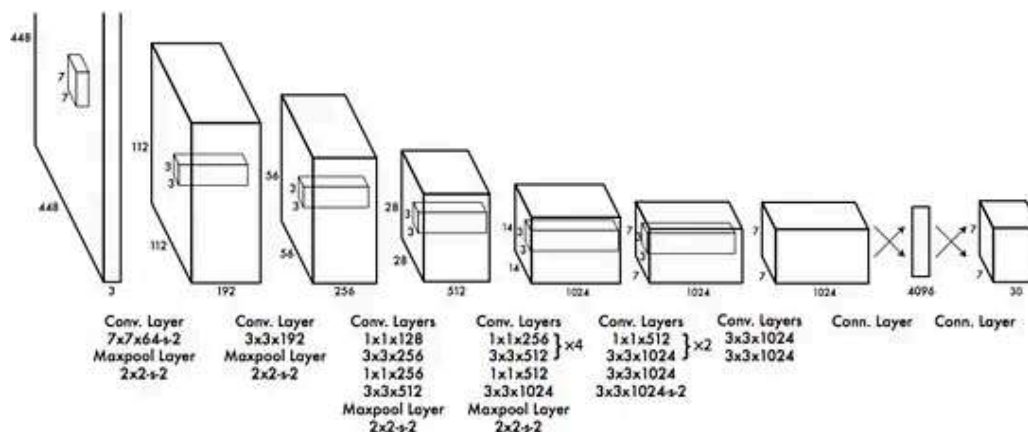


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

We optimize for sum-squared error in the output of our model. We use sum-squared error because it is easy to optimize, however it does not perfectly align with our goal of maximizing average precision. It weights localization error equally with classification error which may not be ideal. Also, in every image many grid cells do not contain any object. This pushes the “confidence” scores of those cells towards zero, often overpowering the gradient from cells that do contain objects. This can lead to model instability, causing training to diverge early on. To remedy this, we increase the loss from bounding box coordinate predictions and decrease the loss from confidence predictions for boxes that don’t contain objects. We use two parameters, λ_{coord} and λ_{noobj} to accomplish this. We set $\lambda_{coord} = 5$ and $\lambda_{noobj} = .5$.

Sum-squared error also equally weights errors in large boxes and small boxes. Our error metric should reflect that small deviations in large boxes matter less than in small boxes. To partially address this we predict the square root of the bounding box width and height instead of the width and height directly.

YOLO predicts multiple bounding boxes per grid cell. At training time we only want one bounding box predictor to be responsible for each object. We assign one predictor to be “responsible” for predicting an object based on which prediction has the highest current IOU with the ground truth. This leads to specialization between the bounding box predictors. Each predictor gets better at predicting certain sizes, aspect ratios, or classes of object, improving overall recall.

Limitations of YOLO

YOLO imposes strong spatial constraints on bounding box predictions since each grid cell only predicts two boxes and can only have one class. This spatial constraint limits the number of nearby objects that our model can predict. Our model struggles with small objects that appear in groups, such as flocks of birds. Since our model learns to predict bounding boxes from data, it struggles to generalize to objects in new or unusual aspect ratios or configurations. Our model also uses relatively coarse features for predicting bounding boxes since our architecture has multiple downsampling layers from the input image. Finally, while we train on a loss function that approximates detection performance, our

loss function treats errors the same in small bounding boxes versus large bounding boxes. A small error in a large box is generally benign but a small error in a small box has a much greater effect on IOU. Our main source of error is incorrect localizations

YOLO9000

YOLOv2: <https://arxiv.org/pdf/1612.08242v1.pdf>

Error analysis of YOLO compared to Fast R-CNN shows that YOLO makes a significant number of localization errors. Furthermore, YOLO has relatively low recall compared to region proposal-based methods. Thus we focus mainly on improving recall and localization while maintaining classification accuracy.

By adding batch normalization on all of the convolutional layers in YOLO we get more than 2% improvement in mAP. Batch normalization also helps regularize the model. With batch normalization we can remove dropout from the model without overfitting.

For YOLOv2 we first fine tune the classification network at the full 448×448 resolution for 10 epochs on ImageNet. This gives the network time to adjust its filters to work better on higher resolution input. We then fine tune the resulting

network on detection. This high resolution classification network gives us an increase of almost 4% mAP

Convolutional With Anchor Boxes. YOLO predicts the coordinates of bounding boxes directly using fully connected layers on top of the convolutional feature extractor. Predicting offsets instead of coordinates simplifies the problem and makes it easier for the network to learn. We remove the fully connected layers from YOLO and use anchor boxes to predict bounding boxes. Using anchor boxes we get a small decrease in accuracy. YOLO only predicts 98 boxes per image but with anchor boxes our model predicts more than a thousand. Without anchor boxes our intermediate model gets 69.5 mAP with a recall of 81%. With anchor boxes our model gets 69.2 mAP with a recall of 88%. Even though the mAP decreases, the increase in recall means that our model has more room to improve.

Fine-Grained Features. This modified YOLO predicts detections on a 13×13 feature map. While this is sufficient for large objects, it may benefit from finer grained features for localizing smaller objects. Faster R-CNN and SSD both run their proposal networks at various feature maps in the network to get a range of resolutions. We take a different approach, simply adding a passthrough layer that brings features from an earlier layer at 26×26 resolution. The passthrough layer concatenates the higher resolution features with the low resolution features by stacking adjacent features into different channels instead of spatial locations, similar to the identity mappings in ResNet. This turns the $26 \times 26 \times 512$ feature

map into a $13 \times 13 \times 2048$ feature map, which can be concatenated with the original features. Our detector runs on top of this expanded feature map so that it has access to fine grained features. This gives a modest 1% performance increase.

During training we mix images from both detection and classification datasets. When our network sees an image labelled for detection we can backpropagate based on the full YOLOv2 loss function. When it sees a classification image we only backpropagate loss from the classification specific parts of the architecture.

Hierarchical classification. ImageNet labels are pulled from WordNet, a language database that structures concepts and how they relate [12]. In WordNet, “Norfolk terrier” and “Yorkshire terrier” are both hyponyms of “terrier” which is a type of “hunting dog”, which is a type of “dog”, which is a “canine”, etc. Most approaches to classification assume a flat structure to the labels however for combining datasets, structure is exactly what we need.

Official website:

<https://pjreddie.com/publications/>

Paper presentation at CVPR 2016:

Other AI articles:

Getting Started with Image Recommendation: A Simplified Approach to Zero-Shot Semantic Similarity using ViT DINO

Building a Q&A bot with LangChain and OpenAI API

ChatGPT, LLMs : Intuition and overview of Instruction finetuning

CLIP (Contrastive Language-Image Pretraining) — Summary and intuition

Deep Learning

Object Detection

Yolo

Machine Learning



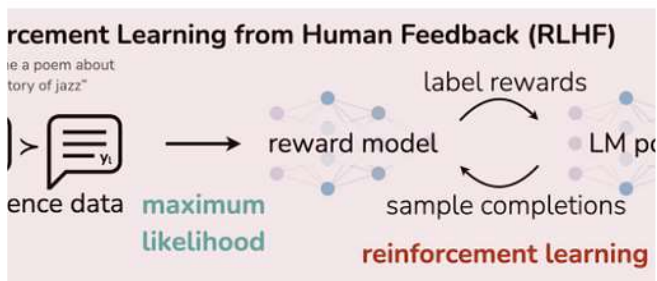
Written by Manish Chablani

2K Followers · Writer for Towards Data Science

Follow

Head of AI @EightSleep , Marathoner. (Past: AI in healthcare @curaiHQ , DL for self driving cars @cruise , ML @Uber , Early engineer @MicrosoftAzure cloud)

More from Manish Chablani and Towards Data Science



Manish Chablani

Aligning LLMs with Direct Preference Optimization (DPO)—...

Direct Preference Optimization (DPO) is a stable, performant, and computationally...

3 min read · Feb 9, 2024



Youness Mansar in Towards Data Science

Meet the NiceGUI: Your Soon-to-be Favorite Python UI Library

Build custom web apps easily and quickly

8 min read · Apr 16, 2024



Rami Krispin in Towards Data Science

Setting A Dockerized Python Environment—The Elegant Way



Manish Chablani

L1 regularization and L2 regularization intuitive explanation

This post provides a step-by-step guide for setting up a Python dockerized developmen...

9 min read · Apr 2, 2024



848



3



70



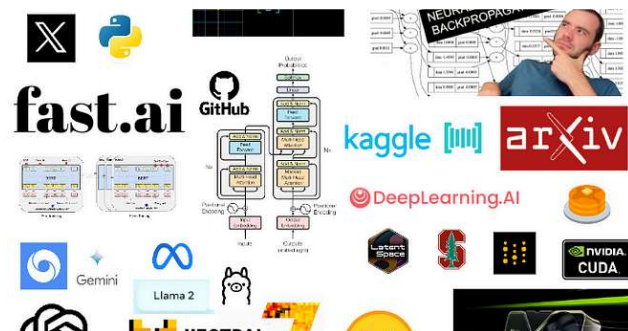
L1 and L2 as forms for regularization are well known and common. While I was explaining...

★ · 2 min read · Sep 29, 2023

See all from Manish Chablani

See all from Towards Data Science

Recommended from Medium



 Benedict Neo in bitgrit Data Science Publication



 VK

Roadmap to Learn AI in 2024

A free curriculum for hackers and programmers to learn AI

11 min read · Mar 10, 2024

 11.1K

 114





YoloV8 Architecture & Cow Counter With Region Based...

Computer vision is an interdisciplinary scientific field that deals with how computer...

8 min read · Nov 21, 2023

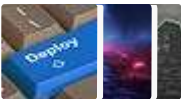
 116

 2





Lists



Predictive Modeling w/ Python

20 stories · 1142 saves



Practical Guides to Machine Learning

10 stories · 1374 saves



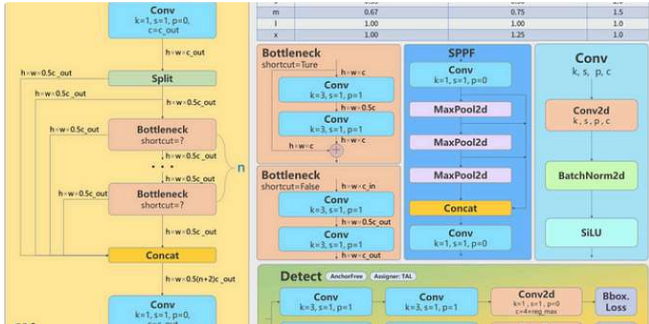
Natural Language Processing

1417 stories · 911 saves



data science and AI

40 stories · 140 saves





Fine-tuning YOLOv8

Step-by-step guide for fine-tuning YOLOv8 using your own datasets in Google Colab

🌟 · 3 min read · Feb 7, 2024



334



3



Juan Pedro

Detailed Explanation of YOLOv8 Architecture—Part 1

YOLO (You Only Look Once) is one of the most popular modules for real-time object...

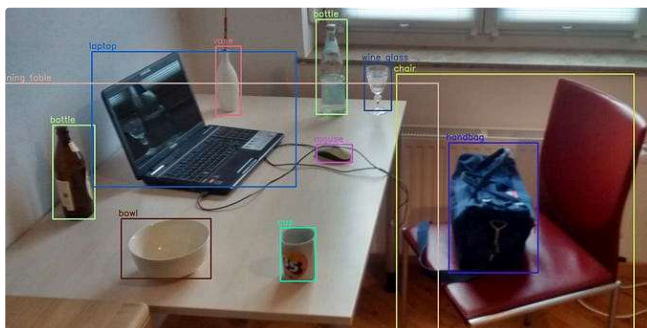
6 min read · Dec 3, 2023



113



1



Francesco Franco in AI Mind

Object Detection with Python and HuggingFace Transformers

(NOTE: This post was deleted, along with several others, during a fit of depression. I...

9 min read · Mar 20, 2024



156



Ikram Noun

self-driving car project, part 2: object detection

Introduction:

6 min read · Mar 25, 2024



6



See more recommendations

[Help](#) [Status](#) [About](#) [Careers](#) [Blog](#) [Privacy](#) [Terms](#) [Text to speech](#) [Teams](#)